

Documentazione DisponibilitaCampoService

Scopo della classe

DisponibilitaCampoService gestisce la logica di business relativa alla gestione delle disponibilità dei campi sportivi (entità **DisponibilitaCampo**). Estende **AbstractService<DisponibilitaCampo, DisponibilitaCampoDto>**, ereditando le operazioni CRUD generiche e aggiungendo metodi specifici per la consultazione delle disponibilità filtrate per data o per singolo campo.

Dipendenze

- **DisponibilitaCampoMapper**: converte le entità **DisponibilitaCampo** nei rispettivi **DisponibilitaCampoDto**.
- **DisponibilitaCampoRepository**: comunica con la base dati per recuperare le informazioni sugli orari e gli slot disponibili.
- **CampoRepository**: consente di verificare l'esistenza delle strutture sportive prima di estrarne le relative disponibilità.

Costruttore

Riceve tramite Dependency Injection il repository generico, il converter, il mapper specifico e i repository dedicati (**DisponibilitaCampoRepository** e **CampoRepository**). Richiama **super(repository, converter)** per inizializzare la superclasse e memorizza le dipendenze nei campi della classe.

Metodi

getAllDisponibilita()

Recupera l'elenco completo di tutte le disponibilità registrate a sistema. Interroga il database tramite **disponibilitaCampoRepository.findAll()** e converte ciascuna entità della lista nel corrispondente DTO utilizzando la Stream API (**disponibilitaCampoMapper::toDTO**).

getDisponibilitaByData(LocalDate data)

Ricerca tutte le disponibilità disponibili per una specifica data solare fornita in input. Interroga il repository con **findByData(data)**. Se la lista restituita dal database è vuota, solleva un'eccezione generale (**Exception**) indicando la mancanza di disponibilità per la data specificata; in caso contrario, converte la lista in DTO e la restituisce al chiamante.

getDisponibilitaByCampo(Integer idCampo)

Restituisce tutte le disponibilità collegate a un determinato campo sportivo identificato da **idCampo**. Prima di accedere alle disponibilità, il metodo esegue un controllo di validazione verificando l'esistenza del campo tramite **campoRepository.findById(idCampo)**, sollevando un'eccezione se il campo non viene trovato. Successivamente recupera gli slot tramite **disponibilitaCampoRepository.findByCampoId(idCampo)** e li converte in una lista di **DisponibilitaCampoDto**.

Flusso di esecuzione

Controller → DisponibilitaCampoService → DisponibilitaCampoRepository / CampoRepository → Database → List<DisponibilitaCampo> → DisponibilitaCampoMapper → List<DisponibilitaCampoDto> → Controller → JSON.

Vantaggi

Separazione delle responsabilità, validazione preventiva delle entità collegate (es. verifica esistenza del campo), gestione formale delle eccezioni per dati non trovati, conversione Entity/DTO centralizzata e sicura, codice isolato e facilmente manutenibile.